

Author

Nimit Bhandari

24f1002045

24f1002045@ds.study.iitm.ac.in

I am Nimit Bhandari, a diploma level student of IITM B.S. Currently I am interested in the field of Data Science and this is my submission of MAD1 Project. Hope you like it!

Description

We need to create a vehicle parking application, for both user and admin side. The admin side will be able to edit, create and delete lots and users will be able to reserve the slots with the vehicle parking number.

AI Usage: Backend (Flask + DB): 25% , Frontend (HTML/JS)+Styling+Charts: 35% , Auth+CRUD Logic : 10%. Mostly AI used in Frontend plus styling.

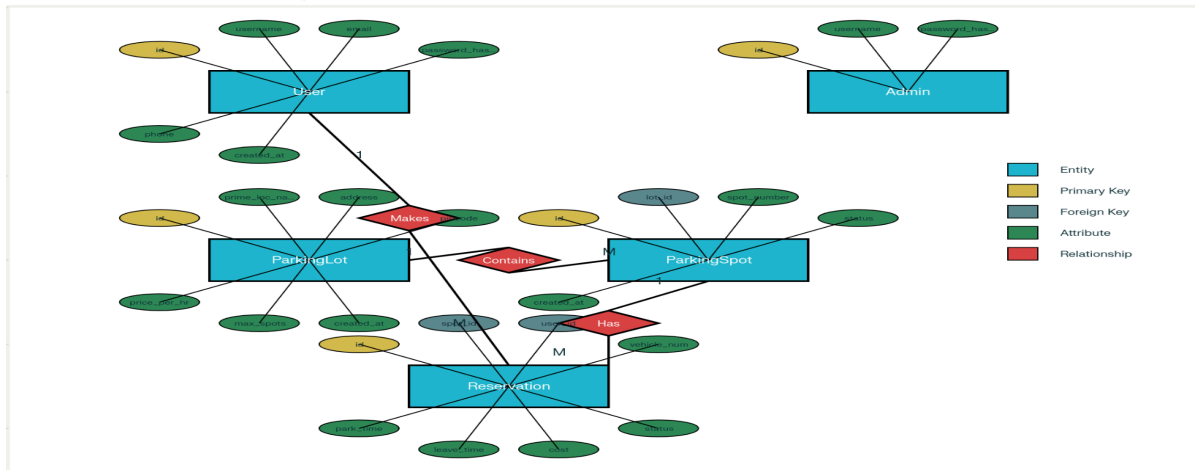
Technologies used

- Flask Framework serves as the main web application framework for building the parking management system, handling HTTP requests, responses, routing, and server-side logic as required by the MAD course curriculum.
- Flask-SQLAlchemy provides a database abstraction layer that simplifies interactions with the SQLite database by allowing Python object operations instead of raw SQL queries while managing complex model relationships between Users, Admins, ParkingLots, ParkingSpots, and Reservations.
- Werkzeug handles security features including password hashing through `generate_password_hash` and `check_password_hash` functions, ensuring secure user authentication for both regular users and administrators.
- Jinja2 Template Engine enables server-side template rendering for dynamic HTML generation with template inheritance capabilities using `{% extends %}` and `{% block %}` syntax for maintainable code structure.
- Bootstrap CSS Framework provides responsive web design with pre-built UI components including cards, forms, buttons, modals, and navigation elements, along with a flexible grid system that makes the application easy to style and mobile-friendly.
- Chart.js delivers interactive data visualization capabilities for parking occupancy statistics, creating pie charts that display available versus occupied parking spots for real-time analytics.
- SQLite Database functions as a lightweight, file-based relational database that stores all application data including user information, parking lots, individual spots, and complete reservation history without requiring separate server installation.
- SQLAlchemy Core with the `func` module enables advanced database operations including aggregate functions like COUNT and SUM for occupancy calculations, as well as complex JOIN operations for analyzing data relationships across multiple tables.

DB Schema Design

- 1. User Table
id (PK, AUTO_INCREMENT), username (UNIQUE, NOT NULL), email (UNIQUE, NOT NULL), password_hash (NOT NULL), phone.
- 2. Admin Table
id (PK), username (UNIQUE), password_hash

- 3. ParkingLot Table
id (PK), prime_location_name, address, pincode, price_per_hour, maximum_spots, created_at
- 4. ParkingSpot Table
id (PK), lot_id (FK), spot_number, status ('A'/'O')
- 5. Reservation Table
id (PK), spot_id (FK), user_id (FK), vehicle_number, parking_timestamp, leaving_timestamp, parking_cost, status ()



Architecture and Features

Architecture

MVC Pattern Implementation: The application follows Model-View-Controller architecture with Flask blueprints separating admin and user modules, utilizing Jinja2 templates for consistent UI rendering and SQLAlchemy ORM for database interactions.

Project Structure: Organized into models (database schemas and defining of database), controllers (the logic of functions and how implemented), templates (UI), and static assets with clear separation. Consist of [App.py](#) which needs to be run to execute the web application.

Core Features

User Management: Secure authentication system with role-based access control, password hashing, and session management for both regular users and administrators.

Parking Management: Complete CRUD operations for parking lots with dynamic spot generation, real-time availability tracking, and flexible hourly pricing models.

Reservation System: Real-time booking with vehicle validation, active reservation management, automatic cost calculation, and comprehensive history tracking.

Admin Dashboard: System overview with occupancy analytics, data visualization through pie and bar charts, parking lot management, and user monitoring capabilities.

Video

<https://drive.google.com/file/d/1ti041NUxJRNvV-hf2akSIBVeIRhPI5zD/view?usp=sharing>